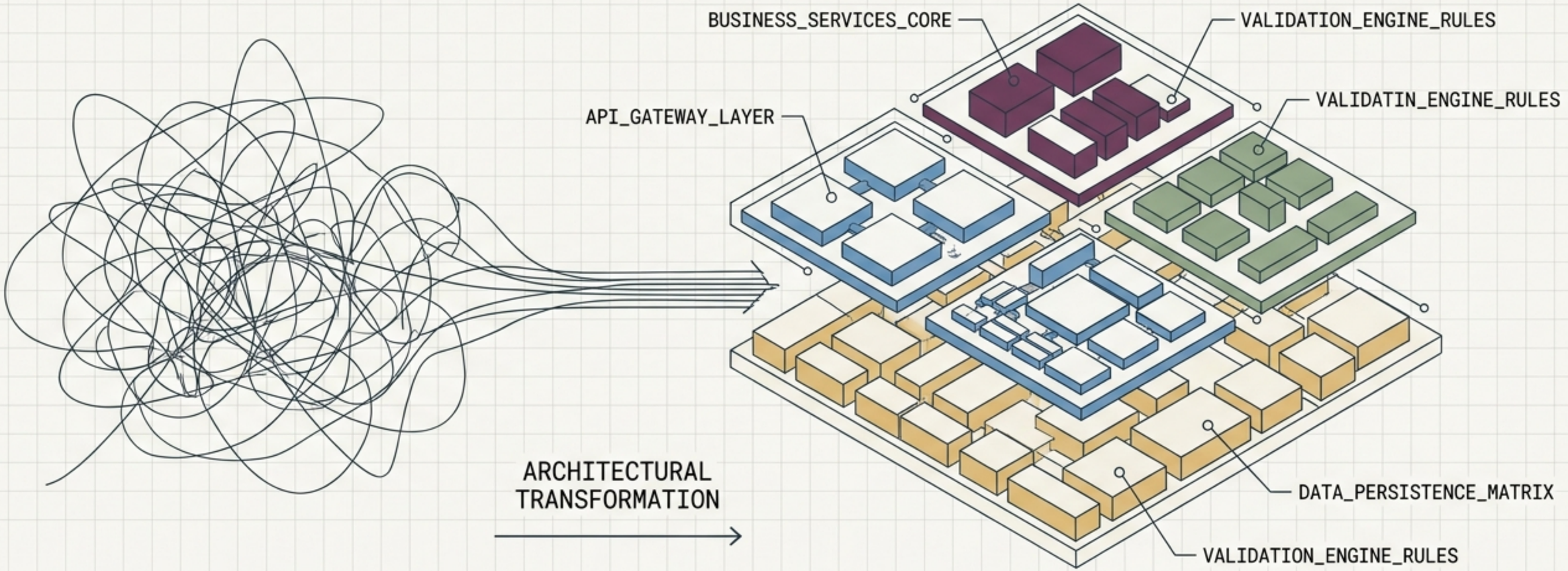
THE STARK HQ ROAMING PROTOCOL

Mastering the Cow Farm Architecture: From Code-Reader to System Architect.



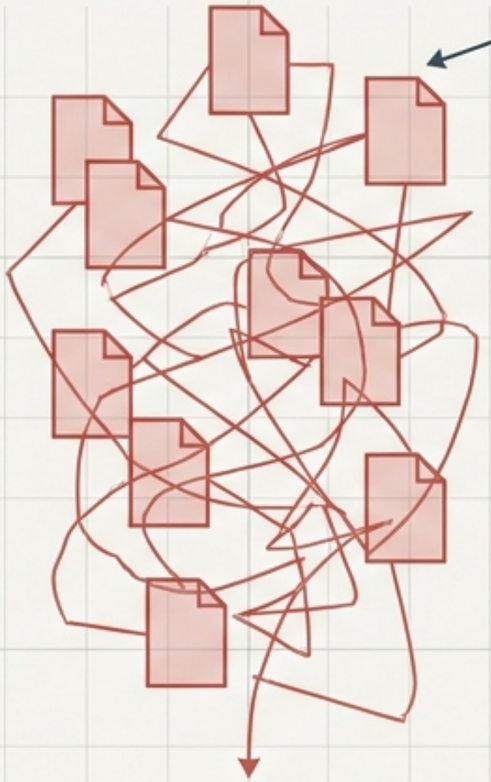
PROTOCOL_V2.0 // SYSTEM_INTEGRATION // ISOMETRIC_BLUEPRINT

Gemini Notebook

Reading an AI codebase file-by-file from top to bottom guarantees you will get buried.

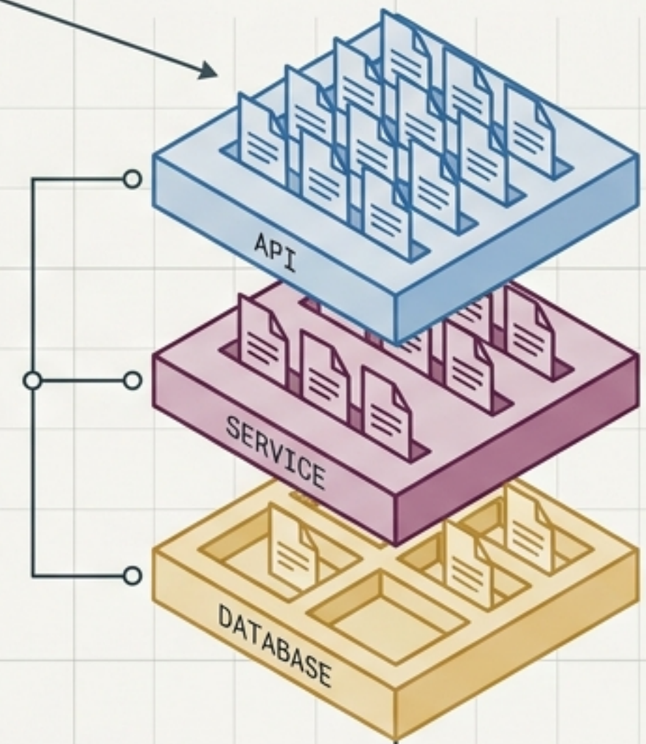
The goal isn't to memorize 40 files. It's to predict where features belong, trace requests, and safely break/change the architecture.

THE SEQUENTIAL TRAP



COGNITIVE OVERLOAD

THE ARCHITECTURAL COMMAND



API

SERVICE

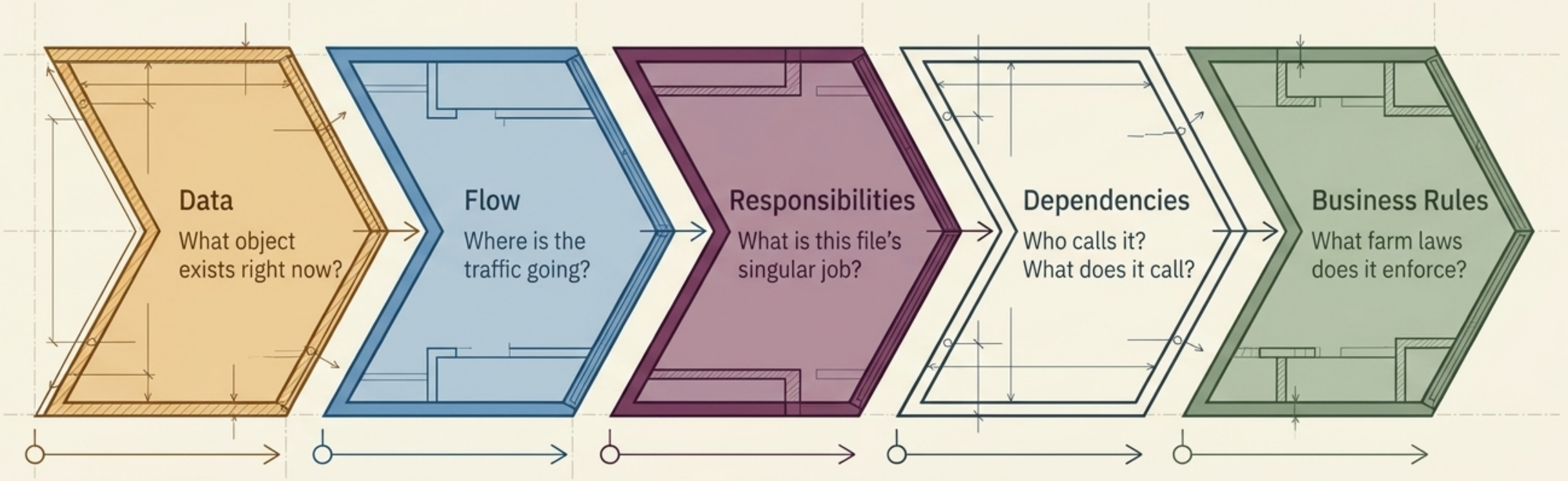
DATABASE

PREDICT, TRACE, MODIFY

Gemini Notebook

Master these five pillars to decode any file instantly.

Once you understand these five, you can open almost any file in the project and immediately orient yourself.



Gemini Notebook

Map the application skeleton before touching the core code.

What is this? → Why does it exist? → What depends on it?

Diagnostic Matrix

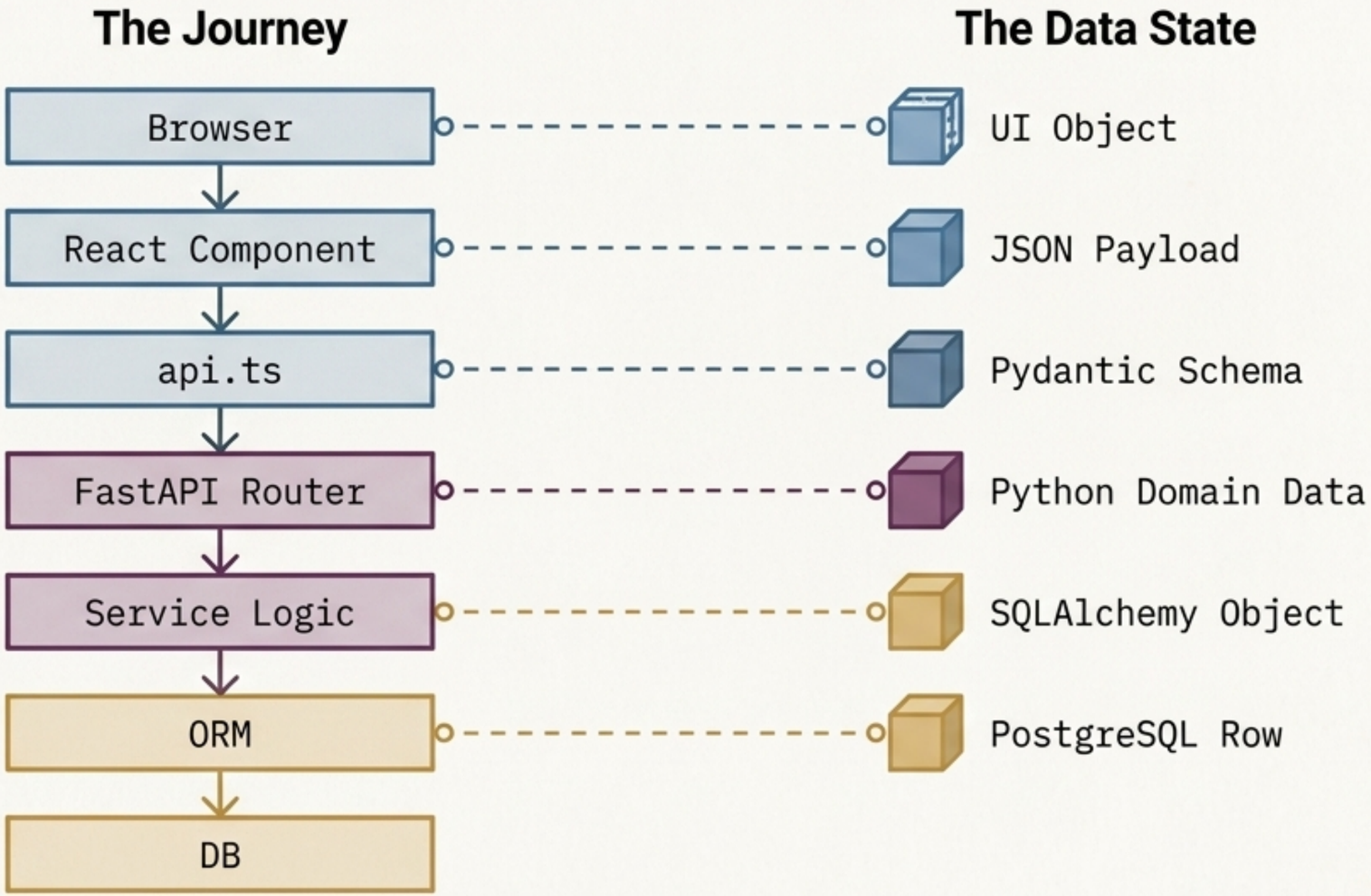
File	Responsibility	Mental Model	Function
`main.py`	Server Assembly	How the server is built	Handles FastAPI app, middleware, lifecycle.
`router.py`	API Traffic Control	Traffic Cop	Handles endpoint grouping, dependency injection.
`config.py`	Environment & Secrets	Where am I running?	Handles DB URLs, settings.
`database.py`	Storage Engine	Commit/Rollback rules	Handles Engine, AsyncSession, Transactions.

Gemini Notebook

Trace a single request to reveal the spine of the application

Example operation: User enters a cow tag + feeding record.

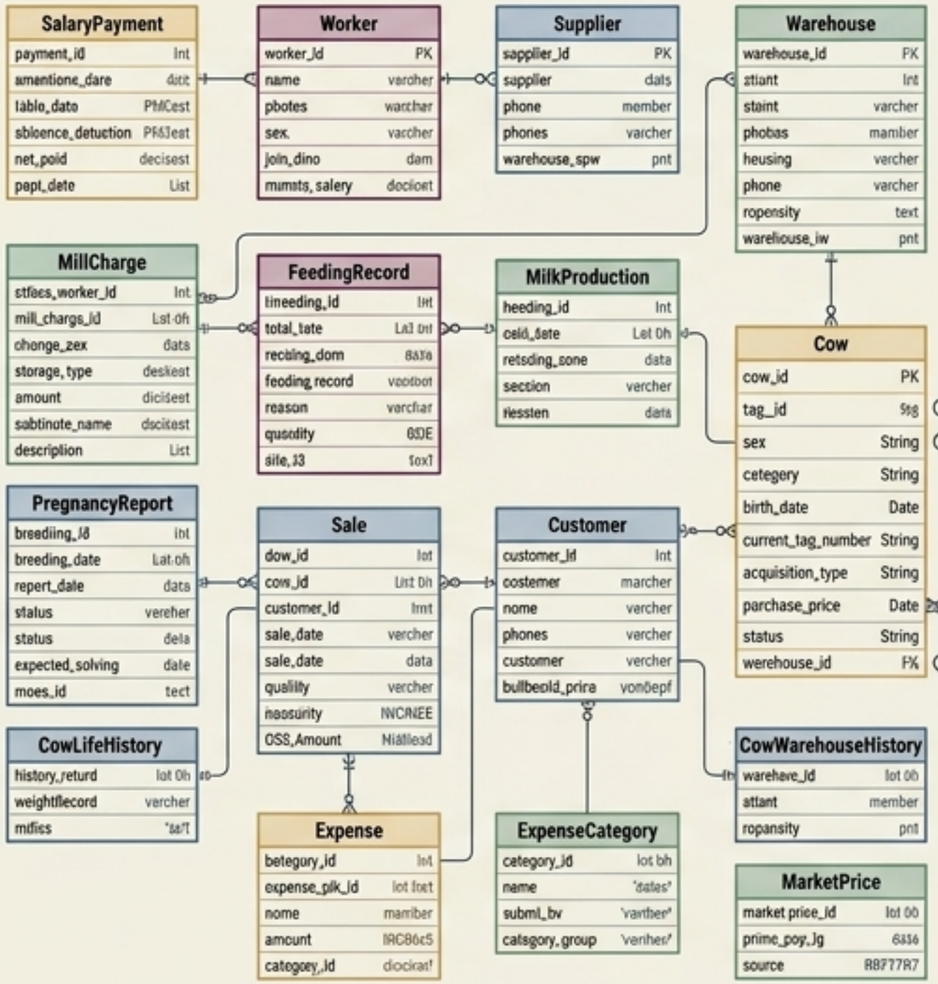
Constant Roaming Question: What exact object exists at this specific point in the flow?



Gemini Notebook

Read domain models as database architecture expressed in Python.

Don’t memorize Python syntax. Ask: Why does this relationship exist?



Primary Key

Columns

Foreign Keys

herd.py

```python
from sqlalchemy import Column, Integer, String, Date, ForeignKey
from sqlalchemy import Column, Integer, String, Boolean, Numeric
from sqlalchemy.orm import relationship
from database import Base

class Cow(Base):
    __tablename__ = "cow"

    id = Column(Integer, primary_key=True, index=True)
    tag = Column(String, unique=True, index=True)
    sex = Column(String)
    category = Column(String)
    birth_date = Column(Date)
    current_tag_number = Column(String)
    acquisition_type = Column(String)
    acquisition_date = Column(Date)
    purchase_price = Column(Numeric)
    status = Column(String)

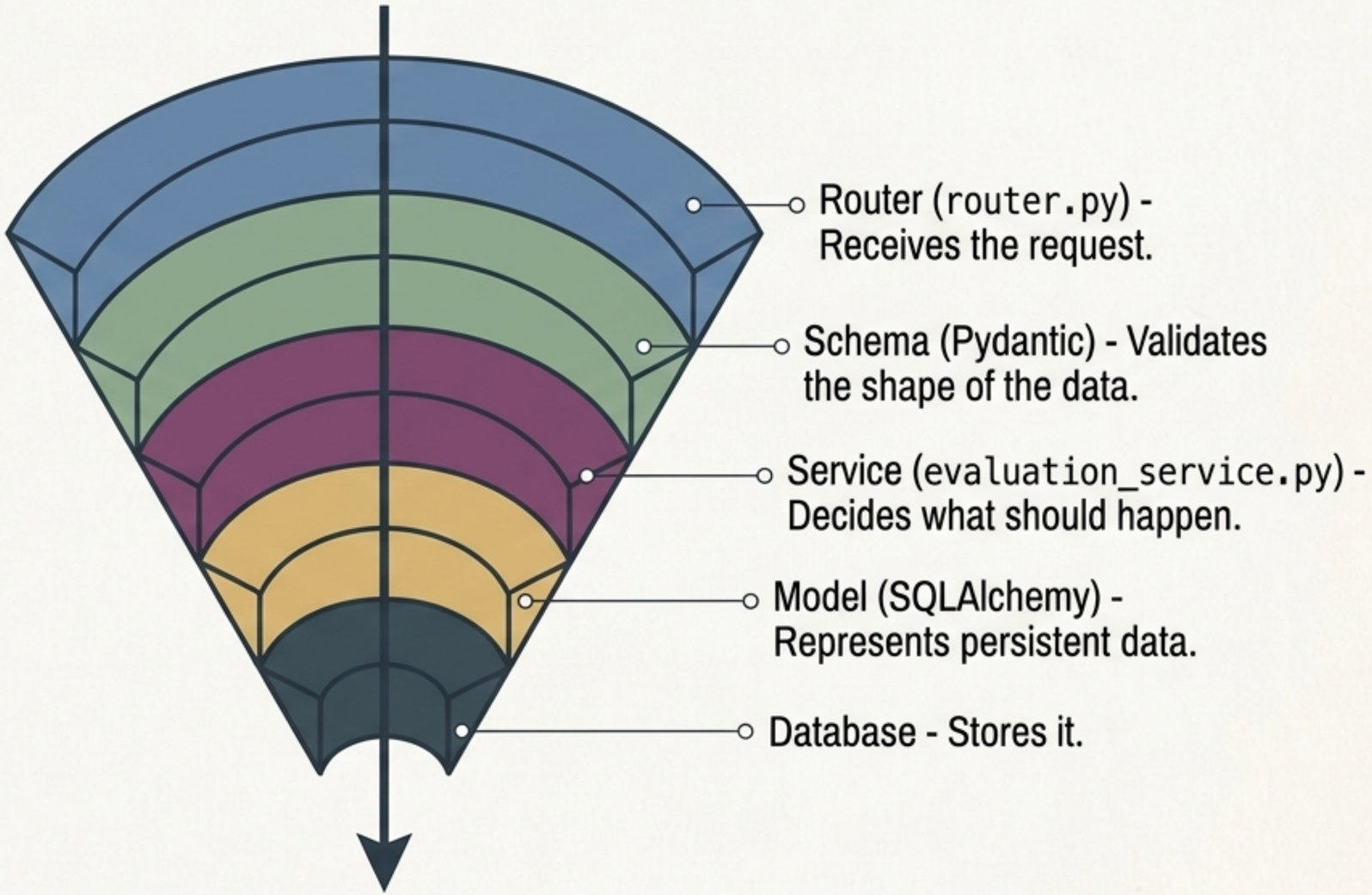
    warehouse_id = Column(Integer, ForeignKey("warehouse.warehouse_id"))
    warehouse = relationship("Warehouse", back_populates="cows")

    # Other relationships...
```

Gemini Notebook

Separate the traffic cops from the business logic.

The critical question: Why isn't this logic inside the API endpoint?



Gemini Notebook

Domain rules are the farm's legal code written in executable form.

`test_domain_rules.py` isn't just QA. It defines the physical boundaries of the farm.

Technical Validation

(Handled by Pydantic/Schemas)

- Is this a number?
- Is this field empty?
- Is this JSON valid?

Business Rules

(Handled by Services / `test_domain_rules.py`)

- Can this cow be sold?
- Can this evaluation be accepted?
- What happens if a record is invalid?

Gemini Notebook

The frontend's role is to participate in the broader system mechanics.

Focus on identifying how the backend knows which user is making the request.



1. Login → Token Generated

2. `AuthContext.tsx` holds React State

3. User interaction triggers event

4. `api.ts` makes call with Auth header

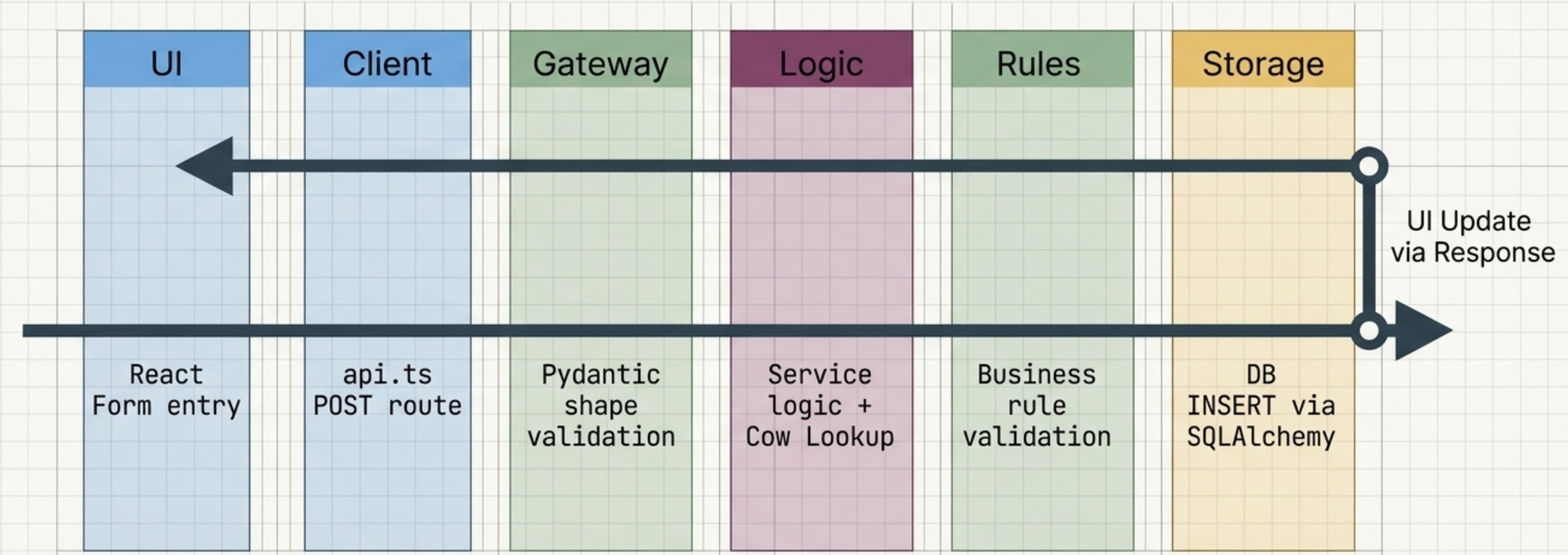
5. `auth.py` verifies current user authorization

6. Backend responds → React State updates → UI refreshes

Gemini Notebook

The Full Vertical Slice: The system stops being 40 files and becomes one machine.

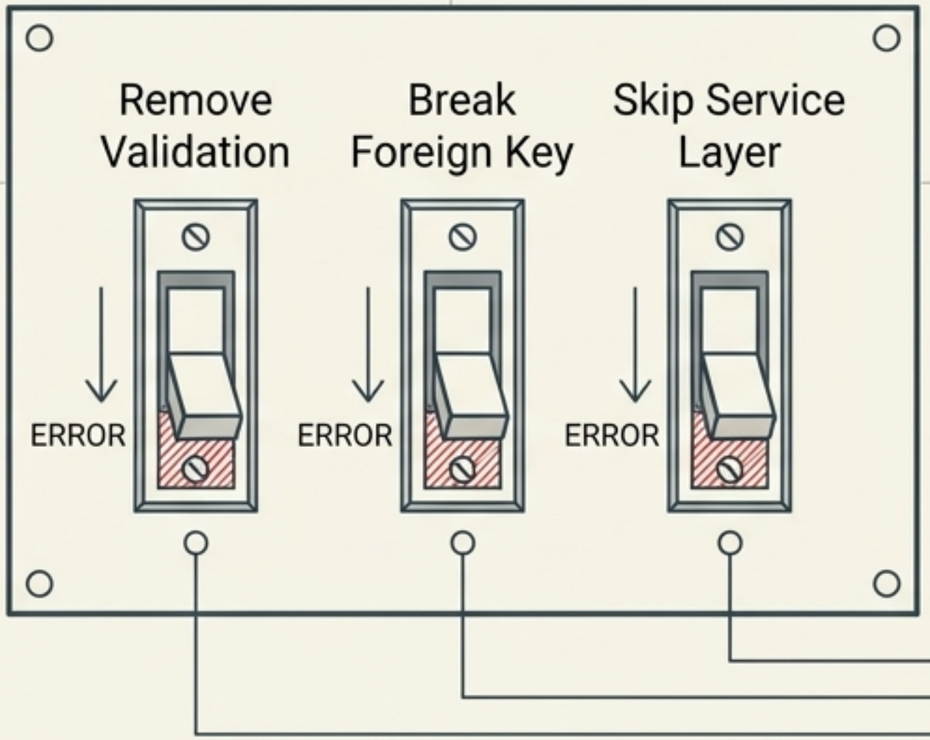
This forensic walkthrough is the fastest way to map the entire architecture in your head.



Gemini Notebook

Break the machine to map its boundaries.

Intentional sabotage and observation is dramatically faster than passively reading explanations.



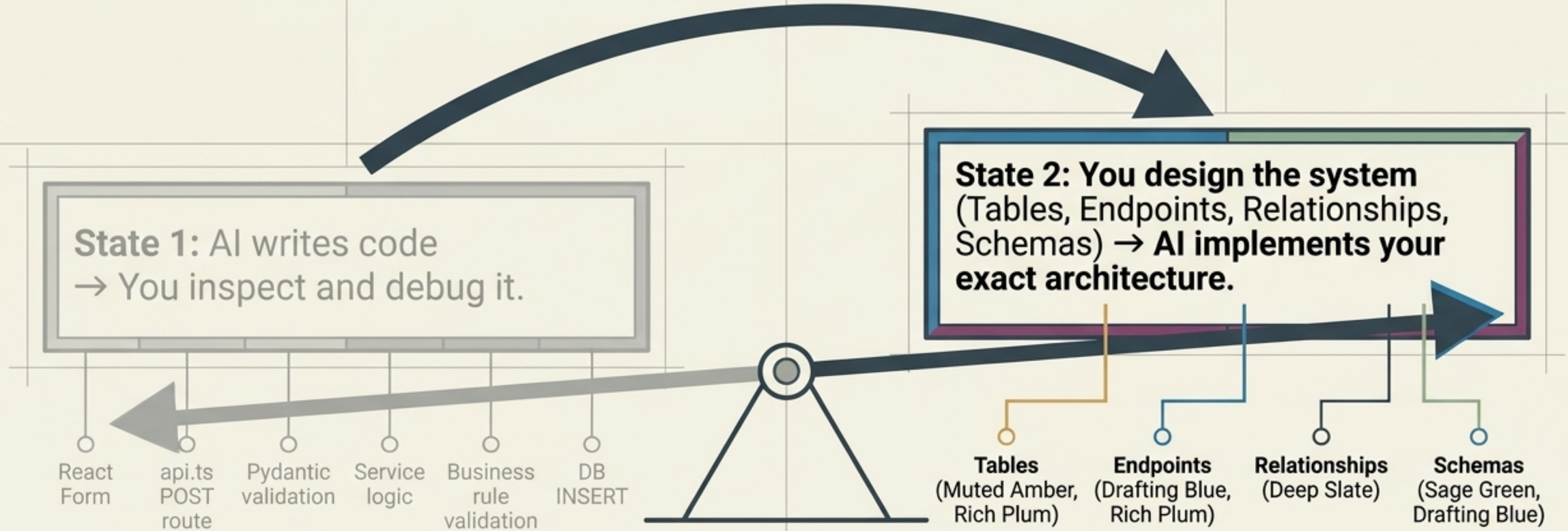
Observation Terminal

- **What breaks?**
- **Where does it break?**
- **Why?**
- **Which layer caught it?**

Gemini Notebook

The true destination is reversing the flow of architecture.

Requirement: Add cow vaccination tracking. You dictate the tables, endpoints, and frontend integration. The AI acts as your compiler.



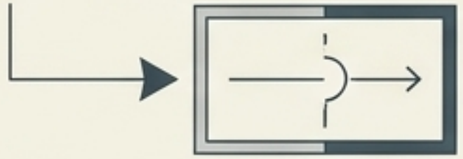
Gemini Notebook

The Daily Rhythm: Your protocol for encountering any new file.

Treat this project as a software-engineering laboratory, not just a generated application. Ask: What did the AI build, and would I design it this way?

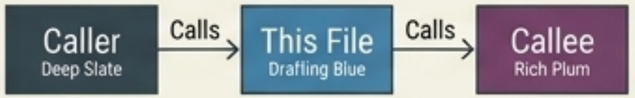
1. Headline

What is its singular job?



2. Architecture Position

Who calls it? What does it call?



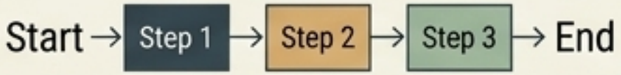
3. Key Objects

Identify schemas, models, and dependencies.



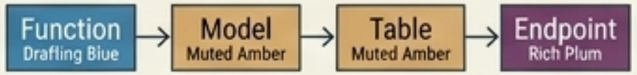
4. Execution

Walk through in runtime order, not file order.



5. Connections

Function → Model → Table → Endpoint.



6. Break Test

Change something. Predict the break.



Gemini Notebook